Device And User Identifiers In iPhone Network Traffic: Activation, Attestation And Anisette

D.J.Leith
Trinity College Dublin, Ireland
17 October 2026

***Abstract*—We describe the device and user identifiers that an iPhone sends to Apple, and how values that are not themselves hardware identifiers are linked back to the handset. Using decrypted network traffic from a jailbroken iPhone 8 (iOS 16.7.11), static analysis of the Apple binaries on the handset and instrumentation of the running processes, we trace each identifier to its origin. Hardware identifiers (the UDID, serial number, IMEI, MEID, ICCID and secure element ID) are sent in clear in many connections. Device activation, Apple's attestation service and two anisette provisioning exchanges create further persistent identifiers: two anisette machine identifiers that did not change during our measurements, a push token, and attestation certificates in which Apple writes up to nine hardware identifiers. We give the endpoints, message formats and encodings of each exchange, including the compressed and signed forms that a simple search for known values misses, so that researchers can recognise these identifiers in their own traces.**

I. Introduction

Network measurement studies of mobile handsets need to know which transmitted values identify the device or the user. Some are obvious, e.g. a serial number in a header. Others are opaque values created by the operating system, e.g. the iPhone's anisette headers, and whether these act as identifiers depends on how they are created. We trace the identifiers that an iPhone sends to Apple back to their origin. Our main findings are:

1. *Hardware identifiers are sent in clear.* The Unique Device Identifier (UDID) is sent under at least nine names. The UDID, serial number, IMEI, MEID, the SIM's ICCID and the secure element ID (SEID) are sent in clear, i.e. unencoded inside TLS, to the Apple ID, App Store, iCloud and iMessage/FaceTime servers. On this handset the UDID is a SHA-1 hash of the serial number, ECID and the two MAC addresses, and so is itself a fixed hardware identifier.
2. *Provisioning creates further persistent device identifiers.* The handset has two anisette machine identifiers, `X-Apple-I-MD-M` and `X-Apple-AMD-M`. Both are produced by the `adid` daemon from provisioning state that works only with the handset's UDID, and neither changed during our measurements: `X-Apple-I-MD-M` from February to October, `X-Apple-AMD-M` from February to June.
3. *Attestation certificates spell out the hardware identifiers.* Certificates issued by Apple's Basic Attestation Authority (BAA) embed the serial number, UDID, ECID, IMEI, MEID, SEID and both MAC addresses, including identifiers that the handset did not send in the request. The handset sends these certificates to the Apple ID, iCloud, Key Transparency and App Store servers. A certificate is also sent with every request for iCloud Private Relay access tokens; it carries the IMEI, MEID and SEID.
4. *The identifiers are sent together.* The AuthKit machine identifier `X-Apple-I-MD-M` is sent in the same request as the Apple account identifier (DSID) in 1,848 requests and with the UDID in 361, so the server can join device and user without any provisioning record.
5. *Location Services is the exception.* No device or account identifier appears in any connection made by the location services daemons `geod` and `locationd` [1].

Contact email: doug.leith@tcd.ie

A. Measurement Setup

We use an Apple iPhone 8 (`iPhone10,1`, iOS 16.7.11, build 20H360) jailbroken with palera1n, with SSL certificate pinning bypassed and traffic decrypted by `mitmdump`, using the protocol of [2, 1]. We use captures from two experiments: a Location Services experiment (17 to 19 February and 3 to 6 June 2026; 4,765 requests to Apple hosts) and an App Store experiment (5 to 16 March 2026; 3,574 requests). The Location Services experiment also includes App Store use, since the test user installs a VPN app.

We read the handset's identifiers (UDID, serial number, ECID, IMEI, MEID and MAC addresses) over USB with `ideviceinfo` from libimobiledevice, which queries the `lockdownd` service that Finder also uses, and use these values as the reference when searching the traffic. We search every request for them by value: headers, cookies, URL parameters, decompressed bodies, base64-decoded header values and base64 data inside plists, matching numeric identifiers also in binary form. We parse every certificate found in the traffic. We analyse the Apple binaries copied from the handset (the dyld shared cache, `adid` and `mobileactivationd`) and confirm behaviour on the running handset with Frida[1], by hooking `adid`, by calling client functions from inside `akd` and `itunesstored`, and by reading key attributes from the keychain in the daemons that requested BAA certificates. The handset is a test device; identifier values are shortened in what

[1]Frida 17.18.0, with `frida-server` installed from the Sileo package manager.

follows (e.g. ccee...8c). Table I explains the Apple terms and acronyms that we use.

Term	Meaning
ADI	Apple Device Information, the obfuscated library (CoreADI on Windows and Android, the adid daemon on iOS) that computes anisette [3]
ADSID	Alternate DSID. A second account identifier, returned at sign-in as X-Apple-Alternate-Id and called altDSID in AuthKit
AMD	The App Store's anisette, sent as X-Apple-AMD-M and X-Apple-AMD; we do not know the expansion
Anisette	Apple's name for the X-Apple-I-MD* device authentication headers
APNs	Apple Push Notification service
ASN.1, DER	ASN.1 is the notation that defines X.509 certificates and their extensions; DER is its binary encoding, in which each value is written as a tag, a length and the content
BAA	Basic Attestation Authority, Apple's certificate authority for Secure Enclave keys (server humb.apple.com, "Humbug")
BIK, BCRT	Basic Identity Key and the certificate that BAA issues for it [4]; Apple's code calls the key RK (reference key)
DSID	Apple account identifier, most likely Directory Services Identifier; Apple does not document the name
ECID	Chip identifier unique to each system-on-chip (UniqueChipID), commonly expanded as Exclusive Chip ID
GrandSlam	Apple's Apple ID authentication service (gsa.apple.com, gsas.apple.com)
ICCID, IMSI	SIM card serial number, and the subscriber identity stored on the SIM
IDS	Identity Services, Apple's registry for iMessage and FaceTime (*.ess.apple.com)
Image4 manifest	The boot manifest that Apple signs for one handset when iOS is installed or updated; it names the device by its ECID
IMEI, MEID	Cellular hardware identifiers of the handset
MID	Machine identifier: X-Apple-I-MD-M or X-Apple-AMD-M
OTP	One-time password: X-Apple-I-MD or X-Apple-AMD
Private Relay	iCloud's two-hop proxy service, run on the handset by networkserviceproxy; its access tokens come from mask-api.icloud.com
RINFO	Routing information, X-Apple-I-MD-RINFO
SEID	Secure Element Identifier, the identifier of the NFC secure element
SEP	Secure Enclave Processor, which holds the handset's private keys
SEQUENCE, SET	ASN.1 containers: a SEQUENCE is an ordered list of values, a SET an unordered one. Other ASN.1 types used here: INTEGER, OCTET STRING (bytes) and IA5String (ASCII text). A context or private tag replaces a value's standard tag with a number chosen by the format's designer, so that fields can be told apart by number
SIK, SCRT	Silicon Identity Key, a Secure Enclave key created at manufacture, and its factory certificate [4]
spim, cpim	Server and client provisioning intermediate metadata, the two ADI provisioning messages [3]
ptm, tk	Persistent token metadata and trust key, returned by the server to complete ADI provisioning [3]
UDID	Unique Device Identifier
UIK, UCRT	User Identity Key, a Secure Enclave key created at activation, and its certificate [4]

TABLE I: Apple terms and acronyms used in this report. Expansions are Apple's own names where Apple uses them, otherwise the names in common use.

II. Related Work

Mobile OS-level telemetry. There are few measurement studies of the data that mobile operating systems themselves send. Leith [5] measures what iOS and Android send to Apple and Google, and later studies [6, 7] extend this to Android OEMs and Google's apps. More recent work studies the Apple App Store [2] and iOS location services [1]. These studies report identifiers seen in traffic but do not trace opaque values created by the operating system, such as the anisette headers, back to the hardware.

Anisette. Anisette has been reverse engineered so that other platforms can authenticate to Apple's servers. Blackwood-4NT [3] analyses the Windows ADI library and its provisioning messages, and JJTech's scripts and Provision [8, 9, 10] reimplement provisioning by running Apple's Windows or Android libraries. These works choose the device identifier freely; we show that on an iPhone it is the UDID, so the machine identifiers are bound to the hardware.

Activation and attestation. Community documentation describes iPhone activation [11, 12, 13] and the Secure Enclave keys and certificates behind BAA [4, 14]. Apple documents only enterprise device attestation [15]. We add the observed message formats, the identifiers that BAA certificates carry, including those Apple adds itself, and where the certificates are sent.

III. Identifiers And How They Link Back To The Handset

Figure 1 shows how the identifiers that are not hardware identifiers are created, and Figure 2 where each is sent. Table II lists the names under which each appears in traffic and its encoding.

We distinguish three ways in which a transmitted value can be linked back to the handset or user:

1. *Directly.* The value is a hardware or account identifier (UDID, serial number, IMEI, DSID). The UDID is a hash of hardware identifiers: on our handset it equals SHA-1 of the serial number, the ECID in decimal, the Wi-Fi MAC address and the Bluetooth MAC address, concatenated. It never changes.
2. *Through a provisioning record.* The value is created in an exchange with an Apple server during which the handset sent hardware identifiers, so the server can record the association. The two anisette machine identifiers, the BAA certificates and the push token are of this kind (Sections IV to VII).
3. *By being sent together.* Two values sent in the same request can be joined by the server. Table III shows that the provisioned identifiers are routinely sent alongside direct identifiers. For example, X-Apple-I-MD-M is sent with the DSID in 1,848 requests, with the UDID in 361 and with the serial number in 108, and X-Apple-AMD-M with the UDID in 222. Linking therefore does not depend on the provisioning record.

Some identifiers are encoded in a way that hides them from a search for their known values: the ADSID is sent as base64 of <ADSID>:postData in x-apple-i-urlswitch-info, the BAA certificate in

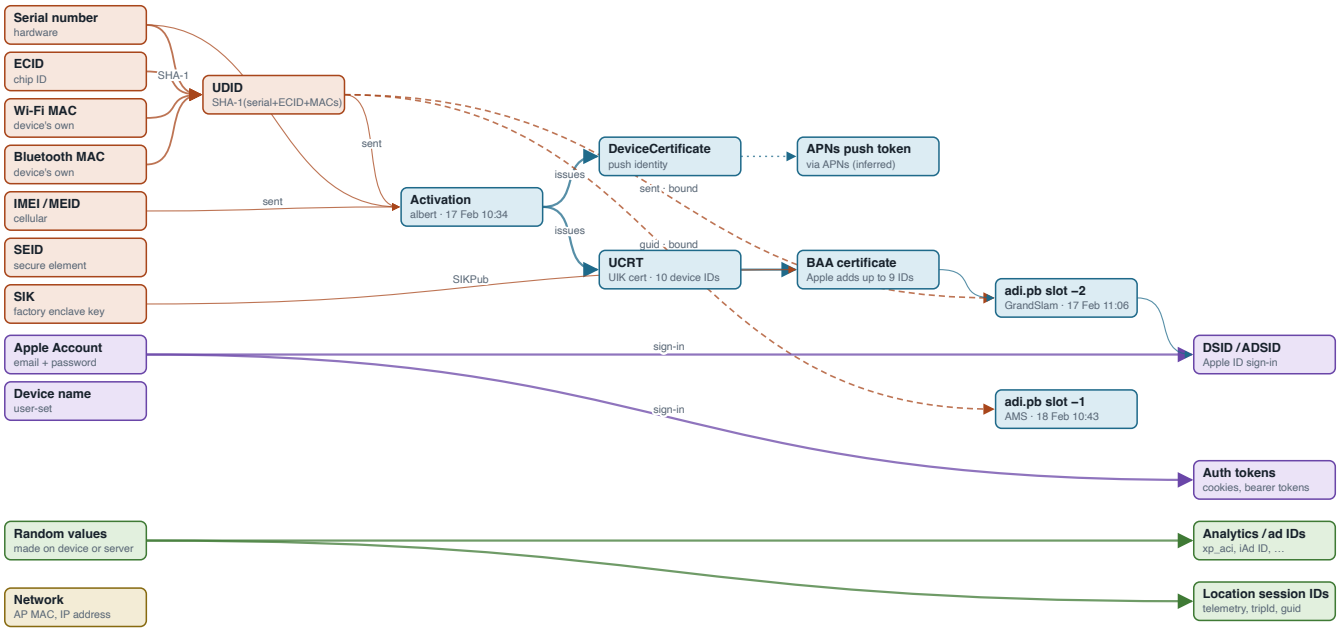


Fig. 1: Where the intermediate identifiers come from, read left to right in time. Thick arrows issue or derive a value, thin arrows show what the handset sent to the issuing step, dashed arrows show a binding that we verified on the handset (the provisioning state only works with this UDID), and the dotted arrow is inferred. Activation, attestation and the two anisette provisioning exchanges each create values that are bound to the hardware identifiers, and at each step the handset sent its serial number and/or UDID.

`X-Apple-Baa` is compressed (Section V), and the hardware identifiers inside certificates are DER-encoded, with the ECID stored as an integer.

IV. Device Activation

After a factory reset the handset cannot complete onboarding until it is activated by Apple, either over a network connection or through a Mac connected by USB. Activation is handled by `mobileactivationd`. We did not capture the activation exchange of our handset: it was activated at 10:34 UTC on 17 February 2026, and our captures start 31 minutes later. We therefore describe the exchange from (i) what the handset received, which we read from the stored activation record, (ii) the endpoints and message fields named in the Apple binaries and (iii) public descriptions [11, 13, 12]. The request contents in this section are inferred, while the received credentials are observed.

Endpoints. The binaries name the following activation endpoints:

- `albert.apple.com/deviceservices/drmHandshake` and `.../deviceActivation` (in `MobileActivation.framework`);
- `albert.apple.com/deviceservices/session`, `.../ucertVend`, `.../certifySB` and `.../baseband` (in `mobileactivationd`);
- `humb.apple.com/humbug/baa` and `tbsc.apple.com/ingest/register` (in `mobileactivationd`).

Activation is a two-step exchange: a handshake that establishes a session (`HandshakeRequestMessage`, `HandshakeResponseMessage`), then the activation request. libimobiledevice implements the same session mode [13]. When a Mac is used it relays the handset's own request over USB (`mobileactivationd` uses `lockdown` and `libReverseProxyDevice`), so the Mac is not a separate authority.

What is sent. Table IV lists the activation request fields named in `mobileactivationd`. Besides the hardware identifiers, the request is built to carry proof that the handset is genuine: a FairPlay signature, BAA attestation, and Secure Enclave key attestation (`mobileactivationd` imports `SecKeyCreateAttestation`). Its entitlement to read protected MobileGestalt values[2] lists the Wi-Fi and Bluetooth MAC addresses, ICCID, MEID, IMEI, ECID, serial number and UDID. The macOS form of the request, which carries the same identifiers in an `ActivationInfoXML` plist signed with a FairPlay certificate, is described in [11].

What is received. The activation record stored on the handset (Listing 1) contains three credentials and FairPlay key data:

[2]MobileGestalt (`libMobileGestalt.dylib`) is the iOS library through which processes query device properties by name, e.g. `MGCopyAnswer("SerialNumber")`. Values that identify the handset, such as the serial number, UDID and MAC addresses, are protected: MobileGestalt returns a protected value only to a process whose code signature carries the private entitlement `com.apple.private.MobileGestalt.AllowedProtectedKeys` and lists that key in it, and returns nothing to any other process. Private entitlements are available only to Apple's own binaries, not to App Store apps.

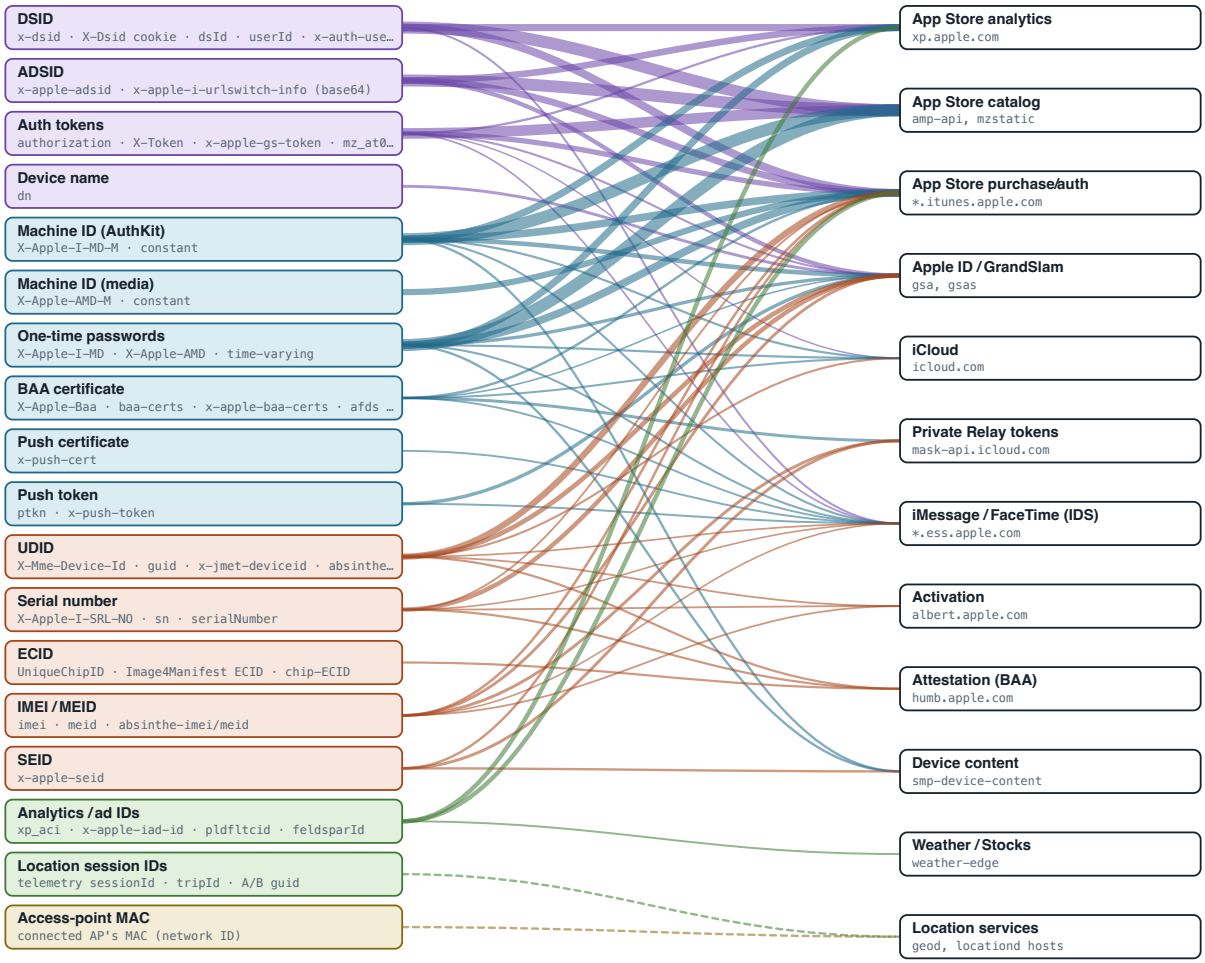


Fig. 2: Where each transmitted value is sent, for both experiments combined. Line width is proportional to the square root of the number of requests. Device, account and provisioned identifiers go to the Apple ID, App Store, iCloud and iMessage/FaceTime servers; the IMEI, MEID and SEID also reach the Private Relay token server, inside a BAA certificate. Location Services connections carry only random session identifiers and the connected access point's MAC address (dashed, from [1]).

1. `DeviceCertificate`, issued by "Apple iPhone Device CA" at activation and valid for three years. The macOS request asks for it as a "Client Push Certificate" [11], and we observe it sent as `x-push-cert` in iMessage/FaceTime registration. It embeds no hardware identifier.
2. `UniqueDeviceCertificate` (UCRT), issued by "FDRDC-UCRT-SUBCA" and valid for seven days. It certifies a Secure Enclave key, the User Identity Key (UIK) [4]. Its subject is the chip identifier and ECID (`00008015-00020C...`), and its extension `1.2.840.113635.100.10.1` contains the board and chip identifiers, ECID, serial number, UDID, IMEI, MEID, both MAC addresses and the SEID (encoded as in Listing 4). We do not observe it in the traffic.
3. `AccountToken`, an old-style text plist signed by Apple, containing the serial number, UDID, IMEI, MEID, IMSI and ICCID together with an activation ticket (Listing 1). It is not sent in the traffic we observe.
4. `FairPlayKeyData`, a 1,594-byte `-----BEGIN CONTAINER-----` block, also not sent.

Listing 1: Structure of the activation record stored by `mobileactivationd` (values omitted).

```
activation_records/activation_record.plist (8,125 B)
DeviceCertificate 1082 B PEM
  issuer CN=Apple iPhone Device CA
  valid 2026-02-17 10:34 to 2029-02-17
UniqueDeviceCertificate 2163 B PEM
  issuer CN=FDRDC-UCRT-SUBCA
  subject CN=00008015-00020C...2E (chip ID-ECID)
  valid 2026-02-17 10:24 to 2026-02-24
AccountToken 1538 B text plist:
  ActivationRandomness ActivationTicket
  IntegratedCircuitCardIdentity (ICCID)
  InternationalMobileEquipmentIdentity (IMEI)
  InternationalMobileSubscriberIdentity (IMSI)
  MobileEquipmentIdentifier (MEID)
  SerialNumber UniqueDeviceID ProductType
  PhoneNumberNotificationURL ActivityURL
  PostponementInfo
AccountTokenCertificate 1241 B
AccountTokenSignature 128 B
FairPlayKeyData 1594 B -----BEGIN CONTAINER-----
RegulatoryInfo 46 B
LDActivationVersion DeviceConfigurationFlags unbrick
```

Later reports to the activation server. We do observe later connections to `albert.apple.com` from `CommCenter`, the cellular daemon, on 18 February and 3 June: `deviceservices/activity/activation` and `deviceservices/activity/phoneNumberSimNotification`.

Identifier	Names in traffic	Encoding	Lifetime	Origin
UDID	`X-Mme-Device-Id`, `guid` (query and body), `x-jmet-deviceid`, `absinthe-udid`, `x-guid`, `device-udid`, `x-appleid-device-udid`, `UniqueDeviceID`, `deviceID` (protobuf)	40 hex digits	fixed	SHA-1(serial ‖ ECID ‖ Wi-Fi MAC ‖ BT MAC)
Serial number	`X-Apple-I-SRL-NO`, `sn`, `serialNumber`, `absinthe-serial-number`, `SerialNumber`	12 characters	fixed	hardware
ECID	`UniqueChipID` and the `ECID` and `UDID` properties of the boot manifests, all in BAA requests (Section V)	integer; chip ID and ECID as 16 bytes	fixed	hardware
IMEI, MEID	`imei`, `meid`, `IMEI`, `absinthe-imei`, `absinthe-meid`, `InternationalMobileEquipmentIdentity`	digits	fixed	hardware
ICCID	`iccid`	digits	SIM	SIM card
SEID	`x-apple-seid`	48 hex digits	fixed	secure element
Device name	`dn`	text (owner's name)	user-set	set at onboarding
Machine ID (AuthKit)	`X-Apple-I-MD-M` (header, and in the `cpd` of GrandSlam requests)	base64, 60 bytes	constant since provisioning	`adi.pb` slot[a] -2, bound to UDID
Machine ID (media)	`X-Apple-AMD-M`, `mid` (JSON response)	base64, 60 bytes	constant since provisioning	`adi.pb` slot[a] -1, bound to UDID
One-time passwords	`X-Apple-I-MD`, `X-Apple-AMD`	base64, 28 and 24 bytes	changes with time	same slots
Routing info	`X-Apple-I-MD-RINFO`	decimal, `50660608`	constant	GrandSlam provisioning
BAA certificate	`X-Apple-Baa`, `baa-certs`, `x-apple-baa-certs`, `afds`, body of `mask-api` token requests	see Section V	1 month to about 333 days	BAA; embeds up to 9 hardware identifiers
Push certificate	`x-push-cert`	base64 DER	3 years	activation
Push token	`ptkn`, `x-push-token`	64 hex digits; base64	at least 81 days	APNs (inferred)
DSID	`x-dsid`, `X-Dsid` cookie, `dsId`, `userId`, `x-auth-user-id` (`D:`DSID), cookie names `mz_at0-`DSID, `mt-tkn-`DSID, `dSID` (protobuf integer)	11 digits	account	Apple ID sign-in
ADSID	`x-apple-adsid`, `x-apple-i-urlswitch-info`	`000191-10-`UUID; base64 of ADSID`:`op	account	Apple ID sign-in
Auth tokens	`authorization` (JWT), `X-Token`, `x-apple-gs-token`, `mz_at0-`, `mt-tkn-` cookies	opaque	rotate	Apple ID sign-in
Analytics, ad IDs	`xp_aci`, `x-apple-iad-id`, `pldfltcid`, `feldsparId`	opaque, UUID	days to months	random or server-assigned

[a] `adi.pb` is the file in which the `adid` daemon stores the state from the anisette provisioning exchanges (Sections VI and VII). The two anisettes are kept apart in it: a caller selects one by the number it passes to `adid`, -2 for AuthKit's anisette, provisioned via GrandSlam, and -1 for the App Store's. We call the state for each number a slot.

TABLE II: Identifiers observed in requests to Apple servers, the names under which they appear, their encoding, lifetime and origin. The table shows that (i) hardware identifiers appear under many names, (ii) several values that are not hardware identifiers are nevertheless bound to the handset by provisioning, and (iii) two account identifiers are sent in encoded or renamed forms.

Listing 2: Report sent to the activation server by `CommCenter` (2026-02-18 13:00 UTC).

```
POST https://albert.apple.com/deviceservices/activity/activation
User-Agent: CommCenter/10307 CFNetwork/1410.1 Darwin/22.6.0
Content-Type: application/x-www-form-urlencoded

Carrier: 11 CarrierName: Carrier Country: 272
InternationalMobileEquipmentIdentity: 3548...71
InternationalMobileSubscriberIdentity: 2721101
MobileEquipmentIdentifier: 3548...47
SIMGID1: 0affffffffffffffffffffffffffffffffffffff
SerialNumber: F4GW...6D
UniqueDeviceID: ccee...8c
```

Each sends the hardware and SIM identifiers as a plist (Listing 2).

V. Device Attestation

Apple's Basic Attestation Authority (BAA, server `humb.apple.com`, "Humbug") issues certificates for keys that the handset records as Secure Enclave keys. Each service that signs its requests obtains its own certificate: we observe nine issuance requests on 17 February and one on 16 March, the first at 11:06:04, one second before the anisette provisioning in Section VI. The certificate is then sent to the service, together with signatures made with the certified key. According to [14, 4], the Secure Enclave creates a new key and attests it with the UIK, the key that the activation certificate (UCRT) certifies. Humbug then traces this attestation back to manufacture: from the UIK to the UCRT that Apple issued at activation, from there to the factory certificate (SCRT) of the Silicon Identity Key (SIK), and from that to the device's manufacturing record. We can

	I-MD-M	AMD-M	UDID	Serial	DSID	ADSID	Push token
I-MD-M	*2000*	234	361	108	1848	1811	57
AMD-M	234	*242*	222	5	230	162	0
UDID	361	222	*390*	124	238	253	54
Serial	108	5	124	*128*	7	95	53
DSID	1848	230	238	7	*1926*	1721	5
ADSID	1811	162	253	95	1721	*1814*	51
Push token	57	0	54	53	5	51	*57*

TABLE III: Number of requests that carry both identifiers, both experiments combined (diagonal: requests carrying the identifier). The anisette machine identifiers travel with the account and hardware identifiers in hundreds to thousands of requests.

Group	Fields
Identifiers	`SerialNumber`, `UniqueDeviceID`, `UniqueChipID` (ECID), `InternationalMobileEquipmentIdentity` and `...Identity2`, `MobileEquipmentIdentifier`, `WifiAddress`, `BluetoothAddress`, `PhoneNumber`, `PhoneNumber2`, `SIMStatus`
Device	`ProductType`, `ModelNumber`, `RegulatoryModelNumber`, `BoardId`, `ChipID`, `BuildVersion`, `ProductVersion`
Proof	`FairPlayCertChain`, `FairPlaySignature`, `DeviceCertRequest`, `BAACertification`, `BAASignature`, `BasebandRequestInfo`, `CollectionBlob`, `FDRBlob`
Session	`HandshakeRequestMessage`, `HandshakeResponseMessage`, `ActivationInfoXML`, `ActivationRandomness`

TABLE IV: Activation request fields named in `mobileactivationd`. The request contents are inferred from the binary, since we did not capture the activation exchange.

confirm only the handset's side of this: the request carries the UIK's public key, which equals the public key of the handset's UCRT, and the SIK's public key. The two requests from `nfcd` carry no UIK key. The UCRT is valid for only seven days and expired on 24 February, yet the request on 16 March still names its key and Humbug issued a certificate, so Humbug does not require an unexpired UCRT; presumably it checks the UIK against its own records.

Where the key is held. Apple does not document BAA. The account in [14, 4] comes from reverse engineering, and Apple's BAA client, DeviceIdentity.framework, calls the key a Secure Enclave reference key (`validSecureEnclaveReferenceKey`) and attests it to the SIK.

We checked this on the handset. Using Frida, we listed the keychain keys visible to each daemon that had requested a certificate (`SecItemCopyMatching`, which only reads). For three of the ten captured requests the key is still in the keychain, and its public key equals `RKCertificationPub` in the request: `2bit-identity-rk-ucrt` for `devicecheckd`, `fpdi-rk-ucrt` for `fairplaydeviceidentityd` and `com.apple.Transparency-rk-ucrt` for `transparencyd`. The keys for the other requests have since been replaced. Every such key has the token ID `com.apple.setoken:aks`, the AppleKeyStore instance of the Secure Enclave token, so the keychain holds only a reference to the key. The software keys stored alongside them have no token ID. The `devicecheckd` key is the one whose certificate is sent in `X-Apple-Baa` during anisette provisioning (Section VI). This shows that the operating system records the BAA keys as Secure Enclave keys, but not where the private key is physically held.

Listing 3: BAA certificate request and response (2026-02-17 11:06 UTC).

```
POST https://humb.apple.com/humbug/baa
User-Agent: iOS Device Activator (MobileActivation-834.140.3.700.2) -
devicecheckd
x-jmet-deviceid: ccee...8c <- UDID
Content-Type: application/xml
<plist> RKCertification: <data 820 B> DER, 30 82 03 30 02 01 02 ...
      RKProperties: <data 1218 B> XML plist:
         AppID: com.apple.devicecheckd BoardId: 2 ChipID: 32789 (0x8015)
         CACert: 0 CertType: 0 SecurityDomain: 1 scrtAttestation: 0
         OIDSToInclude: [1.2.840.113635.100.10.1]
         OsBuildVersion: 20H360 Validity: 479002 (minutes)
         SerialNumber: F4GW...6D UniqueChipID: 5765...98
         SIKPub: <65 B> 04 29 0d ... UIKPub: <65 B> 04 e4 af ... (= UCRT key)
         RKCertificationPub: <65 B> 04 fe 8e ... (= key of issued certificate)
      RKPropertiesSignature: <data 72 B> DER ECDSA signature
HTTP 200 Content-Type: text/html Content-Encoding: gzip
-----BEGIN CERTIFICATE----- (leaf, issuer CN=Basic Attestation User Sub CA1)
-----BEGIN CERTIFICATE----- (Basic Attestation User Sub CA1, issuer ... Root
CA)
```

Request. The request is an XML plist with three data fields (Listing 3). `RKProperties` is itself an XML plist. It carries three public keys: `UIKPub`, which equals the public key of the activation UCRT, `SIKPub`, and `RKCertificationPub`, the new key, which equals the public key of the issued certificate. It also carries the serial number and ECID, and asks for the identifier extension `1.2.840.113635.100.10.1`. The UDID is sent in the `x-jmet-deviceid` header. `RKCertification` is a version 2 DER structure containing a 32-byte hash, a 65-byte elliptic curve public key that matches none of the three keys above and that we could not identify, two 16-byte values and an opaque 668-byte blob, presumably the Secure Enclave's attestation. `RKPropertiesSignature` is a DER ECDSA signature.

Boot manifests. Nine of the ten requests carry the handset's two Image4 boot manifests, `Image4Manifest` (7,308 bytes) and `Cryptex1Image4Manifest`. The first contains the ECID. The second contains a 16-byte `UDID` property made of the chip identifier and the ECID, which is the UDID format of newer iPhones. Both manifests were identical in all nine requests, from 17 February to 16 March, so each identifies the handset in its own right; we found them in no request to any other server.

Certificate. The issued leaf certificate has subject `OU=BAA Certification, CN=<SHA-256 of its own public key>` and is valid from one day before issue for about 333 days, matching the requested `Validity`. Its extension `1.2.840.113635.100.10.1` is a DER `SET` of entries (Table I explains the ASN.1 terms), each a private tag whose number is a four-character code, containing a

Listing 4: Extension 1.2.840.113635.100.10.1 of the BAA certificate (openssl asn1parse, values shortened). The UCRT uses the same encoding and adds `BORD`.

```
SET {
  [PRIVATE 0x43484950] SEQ { IA5 "CHIP", INT 8015 }
  [PRIVATE 'ECID'] SEQ { IA5 "ECID", INT 0x020C...2E }
  [PRIVATE 'bmac'] SEQ { IA5 "bmac", OCT "88:19:08:..." }
  [PRIVATE 'imei'] SEQ { IA5 "imei", OCT "3548...71" }
  [PRIVATE 'meid'] SEQ { IA5 "meid", OCT "3548...47" }
  [PRIVATE 'srnm'] SEQ { IA5 "srnm", OCT "F4GW...6D" }
  [PRIVATE 'udid'] SEQ { IA5 "udid", OCT "ccee...8c" }
  [PRIVATE 'wmac'] SEQ { IA5 "wmac", OCT "88:19:08:..." }
  [PRIVATE 'seid'] SEQ { IA5 "seid", OCT "045C...41" } }
(tag number = the 4 ASCII characters, e.g. 'CHIP' = 0x43484950)
```

`SEQUENCE` of the code as an `IA5String` and the value (Listing 4). The leaf in Listing 3 contains nine entries: chip identifier, ECID, Bluetooth and Wi-Fi MAC addresses, IMEI, MEID, serial number, UDID and SEID. To find which of these values the handset supplies, we searched each request whose certificate has this extension (seven of the ten) for every value in the certificate: in the headers, the body, the nested `RKProperties` and `RKCertification`, in text, hex, raw byte and integer form. Only the chip identifier, ECID, serial number and UDID appear. The IMEI, MEID, MAC addresses and SEID appear in no request, so Humbug adds them itself. Apple already holds them: all of them are in the handset's UCRT, whose key the request names in `UIKPub`. We cannot rule out that the 668-byte opaque blob in `RKCertification` carries them in encrypted form.

Private Relay certificates. Not every certificate uses this extension. The three certificates issued to `networkserviceproxy`, the daemon that runs iCloud Private Relay, are valid for one month and ask for the extensions `1.2.840.113635.100.8.3` and `1.2.840.113635.100.8.7` instead. Extension `100.8.3` is a `SEQUENCE` of seven context-tagged fields. Three of them are the IMEI, the MEID and the SEID, again absent from the request. Two others repeat values from the request: the SIK's public key, and a 32-byte value taken from the `Image4Manifest`, the handset's signed boot manifest, which nine of the ten requests include. We did not identify the remaining 29-byte value and small integer. Extension `100.8.7`, which six of the other seven certificates also carry, holds the iOS build (`20H360`) among other short fields. The two certificates issued to `nfcd` come from a different CA, Basic Attestation System Sub CA1, and carry both kinds of extension.

Use in requests. Each service that obtained a BAA certificate sends it with part of its certificate chain, alongside signatures made with the certified key, in the encodings listed in Table V. We verified each signature with the public key of the BAA certificate sent with it, except for the Key Transparency signature, whose signed content we could not determine.

Private Relay token requests. The Private Relay certificates are sent in the body of `POST mask-api.icloud.com/v1/fetchAuthTokens`, in all 31 such requests between 17 February and 16 March. All 31 requests carry `x-mask-user-tier: FREE`, i.e. the account has no iCloud+ subscription and so no Private Relay, and the user agent "Transparent network proxy for Apple system services"; `networkserviceproxy` presumably fetches the tokens to proxy some of Apple's own system connections. The body is a binary protobuf, although its content type is `application/x-www-form-urlencoded`. We recovered its schema from the Objective-C protobuf (`PBCodable`) classes in Apple's NetworkServiceProxy framework with Extractor [1]. The request is an `NSPPrivacyProxyTokenActivationQuery` with three fields: `authType`, set to `BAA` (the alternative is `ANISETTE`); `baaParameters`, holding `baaSignature`, an ECDSA signature, and the DER `leafCertificate` and `intermediateCertificate`; and `tokenInfo`, holding the `proxyURL` (`cp4.cloudflare.com`, `ie-gmt.exp.fastly-masque.net` or `mask.icloud.com`), the `vendor` (`CloudFlare`, `Fastly` or `Apple_2`), a 32-byte `tokenKeyID` and 20 or 30 `unactivatedTokenLists` entries of 256 bytes each. The response, an `NSPPrivacyProxyTokenActivationResponse`, returns the same number of 256-byte `activatedTokenLists` entries and a `tokenExpiryTimestamp` 1 to 9 days later. With this schema all 31 requests and responses parse with no unknown fields and re-serialise to the same bytes, and `baaSignature` verifies with the certificate's key over the serialised `tokenInfo`. The requests carry no account identifier or account token, so the handset obtains its Private Relay tokens by authenticating as a device, and the server learns the IMEI, MEID and SEID from the certificate. The 256-byte values have the size of RSA-2048 blinded messages, which would make the issued tokens unlinkable to this request when the handset later presents them to a proxy; we did not check this.

VI. Anisette Provisioning Via GrandSlam

Anisette is Apple's name for the `X-Apple-I-MD*` headers that authenticate a device to its Apple ID servers ("GrandSlam") [3, 8]. Before the handset can produce them it must be provisioned. On our handset this happened at 11:06:05 on 17 February, 31 minutes after activation, triggered by Find My (`X-Apple-Client-App-Name: searchpartyd`). The exchange is two requests to `gsa.apple.com/grandslam/MidService` (Listings 5 and 6). Requests and responses are XML plists, although the requests' `Content-Type` header says form encoding. Both requests carry the serial number and the UDID. The UDID, the client information header and a hash of the body are signed with the BAA key of Section V.

The four binary fields are the messages of a provisioning handshake between Apple's server and the handset's Apple Device Information (ADI) library. Apple does not expand their names. The Blackwood-4NT analysis of ADI [3] calls them the server and client provisioning intermediate metadata (`spim`, `cpim`), the persistent token metadata (`ptm`) and the trust key

Header or field	Sent to	Encoding
`X-Apple-Baa`	`gsa.apple.com` (anisette provisioning)	base64(raw DEFLATE(JSON `{"certs": [base64(DER leaf), base64(DER intermediate)]}`)). Raw DEFLATE is the compressed stream of RFC 1951 without the zlib (RFC 1950) or gzip (RFC 1952) header and checksum, so zlib and gzip decoders reject it unless set to raw mode.
`X-Apple-I-Baa-S`	`gsa.apple.com`	base64 of a DER-encoded ECDSA signature (P-256, SHA-256) over the ASCII string `X-Apple-I-Payload-Hash` ‖ `X-Apple-I-Client-Time`, where the payload hash is the hex SHA-256 of the request body.
`X-Mme-Device-Id-S`, `X-MMe-Client-Info-S`	`gsa.apple.com`	base64 of a DER-encoded ECDSA signature over the ASCII value of `X-Mme-Device-Id` (the UDID) and of `X-MMe-Client-Info` respectively.
`baa-certs`, `baa-message`, `baa-signature`	`acsegateway.icloud.com`	`baa-certs`: base64 of the concatenated DER leaf and root certificates. `baa-message`: base64 of `<unix ms>;<URL path>`. `baa-signature`: base64 of a DER-encoded ECDSA signature over the decoded message.
`x-apple-baa-certs`, `x-apple-baa-signature`	`kt-prod.ess.apple.com` (Key Transparency)	certificates as for `baa-certs`; signature header is `<base64 DER ECDSA signature>:<unix ms>` (signed content not determined).
`afds` (plist body field)	`p52-buy.itunes.apple.com/.../buyProduct`	hex string of a DER `SET` containing a signed blob that includes the SEID and a *Basic Attestation System* certificate chain (Sub CA1 and Root) with the same identifier extension.
body of `fetchAuthTokens`	`mask-api.icloud.com` (Private Relay tokens)	binary protobuf `NSPPrivacyProxyTokenActivationQuery`, sent as `application/x-www-form-urlencoded`. Its `baaParameters` hold `baaSignature`, a DER-encoded ECDSA signature over the serialised `tokenInfo` (the token request: `proxyURL`, `vendor`, `tokenKeyID`, 256-byte `unactivatedTokenLists`), and the DER `leafCertificate` and `intermediateCertificate`.

TABLE V: Encodings of BAA certificates and signatures. Each certificate embeds hardware identifiers (nine, or the IMEI, MEID and SEID in the Private Relay certificates), so each of these fields identifies the handset, even where the encoding hides the identifiers from a search for known values.

Listing 5: GrandSlam start of machine provisioning (2026-02-17 11:06:05 UTC).

```
POST https://gsa.apple.com/grandslam/MidService/startMachineProvisioning
User-Agent: akd/1.0 CFNetwork/1410.1 Darwin/22.6.0
X-Apple-Client-App-Name: searchpartyd
X-MMe-Client-Info: <iPhone10,1> <iPhone OS;16.7.11;20H360> <com.apple.akd/1.0
(com.apple.akd/1.0)>
X-MMe-Client-Info-S: MEYCIQC1yQy7...cZw71i <- ECDSA signature over X-MMe-
Client-Info
X-Mme-Device-Id: ccee...8c <- UDID
X-Mme-Device-Id-S: MEUCIH5yh0x2...JrKsw= <- ECDSA signature over UDID
X-Apple-I-SRL-NO: F4GW...6D <- serial number
X-Apple-I-Client-Time: 2026-02-17T11:06:00Z
X-Apple-I-Payload-Hash: 2bdd2f188c9e...8e4b51 <- hex SHA-256 of body
X-Apple-Baa: nVRZj/I2FP0v...7+9Tc= [1684 chars] <- deflated JSON certificate
chain
X-Apple-I-Baa-S: MEYCIQCuXm3A...2c3oZ1 <- ECDSA signature over hash || client
time
Content-Type: application/x-www-form-urlencoded

<plist> Header: {} Request: {} </plist>

HTTP 200 Content-Type: text/x-xml-plist;charset=UTF-8
<plist> Header: {}
      Response: Status: {ec: 0, ed: "", em: ""}
              spim: AAAABAAAAPCe...bhQ/Q= [464 chars] (347 bytes)
              ptxid: 62bf34da-f0e7-43c1-9349-529ac5b2b6bb </plist>
```

Listing 6: GrandSlam end of machine provisioning (headers as in Listing 5).

```
POST https://gsa.apple.com/grandslam/MidService/finishMachineProvisioning
<plist> Header: {}
      Request: cpim: AAAABf+mSruq...ftkoHY [368 chars] (276 bytes) </plist>

HTTP 200
<plist> Response: Status: {ec: 0, ed: "", em: ""}
              X-Apple-I-MD-RINFO: 50660608
              tk: J7apQQdfAmlbnfEqK+bi4g== (16 bytes)
              ptm: AAAABNB73lPJ...pmLA8= [416 chars] (311 bytes)
              ptxid: b809910a-d3ac-4dbd-8fdf-868ebfc343f4 </plist>
```

(`tk`). In the ADI interface that it describes, the handshake runs as follows:

1. The handset calls `startMachineProvisioning`, and the server returns `spim` (347 bytes).
2. AuthKit passes `spim` to `ADIProvisioningStart`, which forwards it to the `adid` daemon and returns `cpim` (276 bytes) and a session handle.
3. The handset sends `cpim` to `finishMachineProvisioning`, and the server returns `ptm` (311 bytes), `tk` (16 bytes) and `X-Apple-I-MD-RINFO`.
4. `ADIProvisioningEnd` passes the session handle, `ptm` and `tk` to `adid`, which writes its provisioning state to `adi.pb`.

From then on `adid` computes the machine identifier and the one-time passwords from this stored state, which works only with the handset's UDID (Section VIII). `spim`, `cpim` and `ptm` begin with a 32-bit big-endian version word (4 or 5), and the rest of each, like `tk`, is high-entropy, so they are presumably encrypted. The last 60 bytes of `cpim` are a copy of 60 bytes of `spim`. `X-Apple-I-MD-M` appears in none of the four messages, so the server does not send the machine identifier; `adid` derives it on the handset. None of the four appears unchanged in `adi.pb`. `X-Apple-I-MD-RINFO` is sent unchanged in later requests.

These four values are sent only once, at provisioning, so they do not identify the handset in later traffic. They create the persistent, UDID-bound state from which every later machine identifier and one-time password is computed.

Use. After provisioning, AuthKit adds four headers to Apple ID and App Store requests: `X-Apple-I-MD-M` (the machine identifier, base64 of 60 bytes), `X-Apple-I-MD` (a one-time password), `X-Apple-I-MD-RINFO`, and, when available, `X-Apple-I-MD-LU`, the SHA-256 of a local user UUID. The one-time password is base64 of 28 bytes: a 32-bit word 5, a 32-bit length 16, the 16-byte password, and a 32-bit word 3. During the Apple ID sign-in on 18 February the same values are also sent inside the request body, in the `cpd` dictionary of `gsa.apple.com/grandslam/GsService2`, together

Listing 7: App Store (AppleMediaServices) anisette provisioning (2026-02-18 10:43 UTC).

```
GET https://auth.itunes.apple.com/auth/v1/otp/1.5/verify
User-Agent: AppStore/3.0 iOS/16.7.11 model/iPhone10,1 hwp/t8015 build/20H360
...
HTTP 200 x-apple-amd-action: SP
       x-apple-amd-data: AAAABAAAAPAk...YTyJc= [464 chars] (347 bytes)

POST https://auth.itunes.apple.com/auth/v1/otp/1.5/finish
Content-Type: application/x-apple-plist
<plist> clientData: AAAABGwB1Z9C...2Sgdg= [348 chars] guid: ccee...8c </plist>
HTTP 200
<plist> settingInfo: AAAABOaJ5VjB...L3GNSn [500 chars] transportKey:
TJbLuae6xvxUfCV20zhB1w== </plist>

GET https://auth.itunes.apple.com/auth/v1/otp/1.5/verify
x-apple-amd: AAAABAAAABCkrkPEaWZ7PKEtwylUH7bE (24 bytes)
x-apple-amd-m: iiYe...BV (60 bytes)
HTTP 200 {"isVerified": "1", "status": 0, "mid": "iiYe...BV"}
```

with the serial number, UDID and the account email address. The value of `X-Apple-I-MD-M` was identical in all 2,000 requests from 17 February to 6 June and in our measurements on the handset on 16 October.

VII. Anisette Provisioning Via The App Store

The App Store, through AppleMediaServices, has its own anisette, sent as `X-Apple-AMD-M` and `X-Apple-AMD`. It is provisioned in a separate exchange, which we observe at 10:43:42 on 18 February, the first time the App Store is opened (Listing 7). The server starts provisioning in a response header rather than at a dedicated endpoint: a `verify` response carries `x-apple-amd-action: SP` and `x-apple-amd-data`, a 347-byte message with the same form as `spim`. The handset answers with `clientData` (the counterpart of `cpim`) and `guid`, the UDID, and receives `settingInfo` and `transportKey` (the counterparts of `ptm` and `tk`). The next request already carries the new headers, and the server echoes the machine identifier back as `mid`.

The App Store one-time password is base64 of 24 bytes: a 32-bit word 4, a 32-bit length 16 and the 16-byte password. `X-Apple-AMD-M` is sent to the App Store purchase and authentication servers (`*.itunes.apple.com`) in 242 requests, always with the same value, and in 222 of them together with the UDID.

FairPlay signatures. The App Store's FairPlay code, which signs purchase requests and produces `kbsync`, also identifies the handset by a hardware identifier. AppleMediaServices fetches this identifier with the function `AMSFairPlayGetHardwareInfo` and keeps it with each FairPlay signing session. We called this function once inside the App Store daemon `itunesstored`: it returns a 20-byte identifier that is exactly the UDID in binary form. The App Store's FairPlay signatures are therefore tied to the UDID as well. `kbsync` is produced by the same FairPlay code, but we did not trace whether it uses this identifier.

VIII. How The Anisette Values Are Computed On The Handset

Where the values come from. Both anisettes are computed by one system daemon, `adid`. The frameworks that send them do not compute them themselves: six system frameworks, including AppleIDSSOAuthentication and AppleMediaServices, each contain a copy of a small, obfuscated client library that forwards each request to `adid`. Its main function, `ADIOTPRequest` (the client symbol `_qi864985u0`), takes a numeric identifier and returns a machine identifier and a one-time password.

Two fixed identifiers. We called `ADIOTPRequest` ourselves from inside the AuthKit daemon `akd`. With the identifier -2 it returns the values sent as `X-Apple-I-MD-M` and `X-Apple-I-MD`. With -1, the identifier that AppleMediaServices uses, it returns those sent as `X-Apple-AMD-M` and `X-Apple-AMD`. Calls with 0 or with the account's DSID fail with error -45061 (not provisioned), so the provisioning state is held under the two fixed identifiers, not under the Apple account: neither anisette is provisioned per account. `adid` keeps the state for each identifier, which we call a slot, in a single 905-byte file, `adi.pb`. The file was last written one second after the App Store provisioning, consistent with each provisioning exchange writing its state to this one file.

What `adid` reads. Tracing `adid` on the handset, we found that it reads only one hardware identifier, the UDID, once when it starts. Its entitlement to read protected MobileGestalt values lists only that value, and it has no code to read hardware identifiers from the kernel's device registry. When it starts it also reads `adi.pb`, makes three calls to the FairPlay kernel driver (`com_apple_driver_FairPlayIOKit`, selectors 22, 23 and 26) and checks whether it is running in a virtual machine. It reads nothing further when computing anisette.

The state is bound to the UDID. To test whether the stored state depends on the UDID, we restarted `adid` with the UDID that it reads replaced by a different value. We also blocked writes to its files, so that the stored state could not change. Both slots then fail with error -45061; with the real UDID restored, both work. The provisioning state therefore works only with the UDID it was created with. This matches the public tools that emulate anisette on other platforms, which choose an arbitrary device identifier before provisioning and must keep it afterwards [10, 8].

What we could not determine. We could not find how `adid` combines the UDID, the FairPlay driver's responses and the server's provisioning data. Its code is obfuscated throughout: control flow passes through computed jumps and constants are hidden, so decompiling it does not recover the algorithm. Tracing it at run time did not show this either. Apart from the start-up call that reads it, the UDID appears in none of the data that `adid` exchanges with other processes, the FairPlay driver or its files, either as text or as an MD5, SHA-1 or SHA-256 hash, so `adid` transforms it within its own code before use.

IX. Limitations

One handset. We measured one handset running one iOS version, so the formats may differ on other models and versions. In particular, the UDID formula that we confirm applies to handsets of this generation. Newer iPhones use a different

UDID: in our captures from an iPhone 12 (`iPhone13,2`), every UDID header carries the 25-character form chip identifier, hyphen, ECID (`XXXXXXXX-XXXXXXXXXXXXXXXX`), the same form that the iPhone 8 uses inside its boot manifest (Section V).

Activation not captured. We did not capture the activation exchange of our handset, so the activation request contents in Section IV are inferred from the binaries and public descriptions.

Persistence across reset. We observed one provisioning of each anisette, so we do not know whether re-provisioning, e.g. after a factory reset, produces a new machine identifier. Since the handset sends its UDID during provisioning, Apple's server could link an old and a new machine identifier in any case.

Opaque payloads. Some payloads are binary formats that we did not decode: the `validation-data` field and `absinthe-*` signature headers of iMessage/FaceTime registration, the App Store's `kbsync`, the signed blob inside `afds`, which the NFC secure element produces (we decoded only its outer structure), `FairPlayKeyData` and the 668-byte Secure Enclave attestation. None has a public format. All are presumably encrypted or signed, by Apple's obfuscated FairPlay-family code or by the Secure Enclave, so decoding them would need the same analysis of obfuscated code that we could not complete for `adid`. They may contain further identifiers, so the identifiers we report are a lower bound.

Linkage, not use. The links that we describe show what Apple's servers can associate; we do not know which associations Apple actually makes.

X. Data And Code Availability

- The decoded network traces of the location services daemons on the same handset, from the experiments of February and June 2026, are at https://github.com/doug-leith/ios-location-services-study-data.
- Extractor and the `ios_decoder` traffic decoder, with the recovered protobuf schemas, including the NetworkServiceProxy schema used in Section V, are at https://github.com/doug-leith/ios-protobuf-decoding.
- The decoded traces of the App Store study, the `mitmdump` decoding add-on and the certificate-pinning bypass used for the captures are at https://github.com/doug-leith/poor-privacy-practices-of-the-apple-app-store.

XI. Use Of AI

This report was drafted by Claude (Anthropic, Claude Opus 5.5), working in Claude Code, which also carried out the analysis reported here under the author's direction. The author reviewed the report.

References

[1] D. Leith, "Decoding What iOS Location Services Send to Apple: An LLM Is an Unreliable Analyst but a Useful Toolmaker," Technical report, Trinity College Dublin, 2026.

[2] ——, "Poor Privacy Practices Of The Apple App Store: Cookies, Advertising and Tracking Of Users," arXiv:2610.05475, 2026. [Online]. Available: https://arxiv.org/abs/2610.05475

[3] A. Ionescu, "Blackwood 4NT: Grand Slam Authentication for Windows NT," https://github.com/ionescu007/Blackwood-4NT, accessed October 2026.

[4] The Apple Wiki, "Activation keys and certificates," https://theapplewiki.com/wiki/Activation_keys_and_certificates, last edited 17 August 2026, accessed October 2026.

[5] D. J. Leith, "Mobile Handset Privacy: Measuring the Data iOS and Android Send to Apple and Google," in *Proc SecureComm, Security and Privacy in Communication Networks, LNICST 399*, 2021, pp. 231–251.

[6] H. Liu, P. Patras, and D. J. Leith, " On the data privacy practices of Android OEMs," *PLoS ONE*, vol. 18, no. 1, p. e0279942, 2023.

[7] D. J. Leith, "What Data Do the Google Dialer and Messages Apps on Android Send to Google?" in *Proc SecureComm, Security and Privacy in Communication Networks, LNICST 462*, 2022, pp. 549–568.

[8] JJTech, "anisette.py," https://gist.github.com/JJTech0130/fa240e86f639c3a3519599417ffccf61, accessed October 2026.

[9] ——, "Apple's GrandSlam Authentication protocol," https://gist.github.com/JJTech0130/049716196f5f1751b8944d93e73d3452, accessed October 2026.

[10] Dadoum, "Provision: set of tools interacting with Apple servers on Linux," https://github.com/Dadoum/Provision, accessed October 2026.

[11] The Apple Wiki, "Albert," https://theapplewiki.com/wiki/Albert, last edited 24 July 2024, accessed October 2026.

[12] M. Mosen, "macdocs: iOS activation," https://github.com/mosen/macdocs/blob/master/source/DEP/ios-activation.rst, accessed October 2026.

[13] libimobiledevice project, "libideviceactivation: a library to manage the activation process of Apple iOS devices," https://github.com/libimobiledevice/libideviceactivation, accessed October 2026.

[14] The Apple Wiki, "Basic Attestation Authority," https://theapplewiki.com/wiki/Basic_Attestation_Authority, last edited 18 August 2026, accessed October 2026.

[15] Apple, "Managed Device Attestation for Apple devices," https://support.apple.com/guide/deployment/dep28afbde6a/web, apple Platform Deployment guide, accessed October 2026.